Freight Rate Model: Validation and Split Approach

Ayush Shekhar | Spotter ML Engineer assessment

1. Summary

The final model is a gradient-boosted tree model on log(rate) for the load-specific part (distance, pickup/delivery location, equipment, weight, quote_signal), plus a small linear term for the market level and a time trend. On out-of-time validation (train on the past, predict the next two months, five times) it averages 2.08% MAPE / $48 MAE, compared with 2.62% / $61 for the same GBM without the linear term. The final model was retrained on all clean January to October loads before predicting November and December. These windows were also what I used to choose the model, so treat the numbers as selection results, not an untouched test (more on that in section 2).

2. How I split and validated the data

train_test.csv runs from 1 Jan to 31 Oct 2025, and validation.csv covers 1 Nov to 31 Dec 2025. So the real job is a forecast one to two months past the last label, and I set the validation up to look the same. A random K-fold split would put loads from the same day on both sides of the split. The model then picks up each day's market level from the training rows and looks better than it really is. I checked this: a random 5-fold CV gave about 1.8% MAPE for a model that scores about 3.0% when tested on later months.

I used a rolling-origin (expanding window) split. For each cutoff from 1 May to 1 Sep, the model is trained on every load before the cutoff and tested on the two calendar months after it. That gives five test windows (May-Jun through Sep-Oct). They cover months where the market was rising, peaking, falling and flat, so a model that only works in one regime shows up.

One caveat on that: every model and feature choice below was made by looking at these same five windows, so their average is a model-selection result and probably a little optimistic. I never tuned anything on validation.csv, so Spotter's scoring of Nov-Dec is the real untouched test. If I did this again I'd keep the Sep-Oct window locked until the very end and only score the final model on it once.

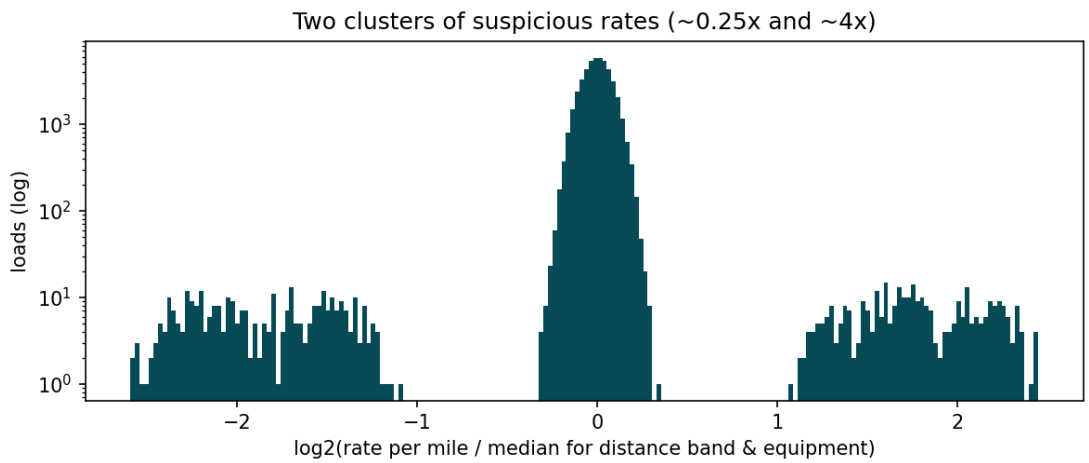
There was one more thing to test. Eight cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) only exist in validation.csv, and about 12% of the validation loads touch one of them. To check that case I removed 8 random cities completely from training (3 repeats), trained on data before September, and compared the error on Sep-Oct loads that touch those cities with loads that don't.

Same-day averages. The model uses two inputs built from all loads on the same date: the mean market_index and the mean quote_signal for that day. Both are averages of input features, never of rates, and in validation they are built the same way (from every load on the test date). That still assumes the whole day's batch of loads is available when predictions are made, which is true here because validation.csv is handed over as one file. If loads had to be priced one at a time, I'd use the per-load variant (each load's own market_index, no daily averages). It scores 2.09% against 2.08%, so the results don't depend on this assumption.

Metrics. MAPE = mean of |predicted - actual| / actual over the loads in a window. MAE is the same in dollars, and RMSE weights big misses more. Bias = mean of log(predicted / actual) x 100, so negative means the model under-predicts. Each number in the tables is a simple average over the five windows. MAPE is the main metric because rates go from about $150 to over $8,000, and a $50 miss means very different things at each end. The suspicious rows from section 3 are left out of the test windows too, so these numbers measure error against realistic rates.

3. Data issues I found and what I did

Issue	Handling
Suspicious target values	677 rows (1.4%) have a rate that is about 0.2-0.4x or 3-5x what the rest of the data says for that distance and equipment, with nothing in between. That pattern points to entry or unit errors rather than real market moves, though I can't confirm it from the data alone. I flag them with an L1-loss GBM (which they don't pull around) where |log residual| > 0.4, and drop them from training. Keeping them in raises out-of-time MAPE on clean rows from 2.08% to 3.88%.
Negative weight	292 train / 145 validation rows. The absolute value behaves exactly like a normal weight, so I use abs(weight).
Capped weight	About 1,200 rows sit at exactly 47,500 lb and about 30 at 5,000 lb. These look like clip limits, so I kept them as they are.
Missing weight	300 / 165 rows. Left as NaN. HistGradientBoosting learns its own branch for missing values.
Missing market_index	374 / 249 rows. Filled with that day's mean index. The row value is basically the daily level plus noise.
Coordinates	Several coordinates are shifted or clipped (Boston and Providence both at lon -69.5, Laredo at lat 25.5, Shreveport and New Orleans only 7 miles apart). Each city still has exactly one coordinate pair, and rates follow the recorded distance column, so I trust distance and use lat/lon only as a relative location.
December chart inputs	This file has only city names and no market columns. I look up coordinates from the city table, and for each date I use that day's mean market_index and quote_signal from the validation loads.

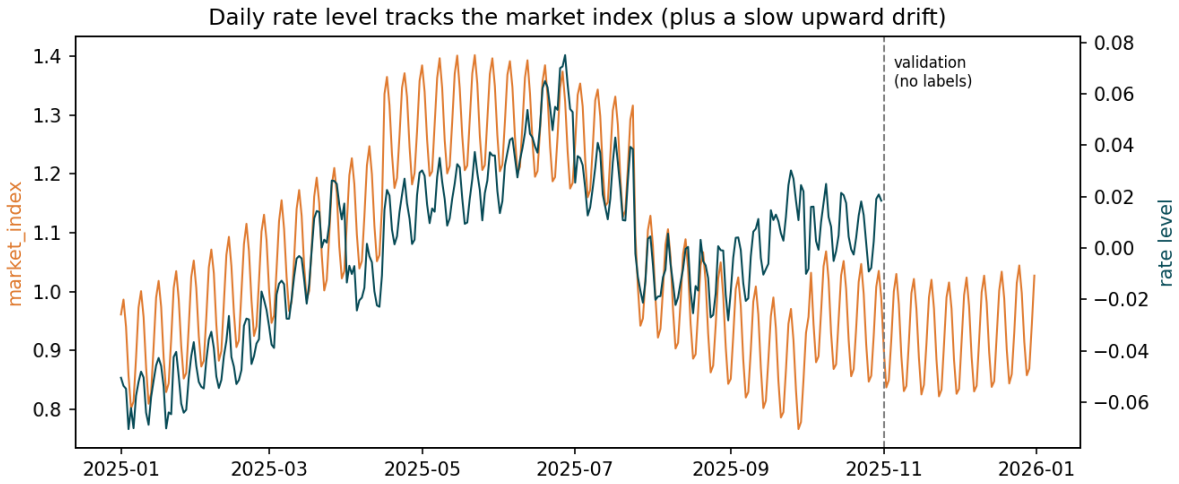


Rate per mile relative to the median for its distance band and equipment. The two small clusters far from 0 are the rows I treat as likely errors.

4. What drives the rate

Distance is most of it. Rate per mile falls from about $2.6 under 250 miles to about $1.8 above 2,000 miles. Flatbed runs about 8% over dry van and reefer about 13%. Weight adds roughly 10% between 5k and 45k lb.

The time pattern is the interesting part. I took the residuals of a model with no time inputs and averaged them per day, giving one 'rate level' number for each of the 304 training days. Regressing that daily level on the same-day mean market_index plus a linear time trend gives R-squared of about 0.80. To be clear, that is 80% of the variation in daily average levels, not 80% of the variance of individual loads. The trend works out to roughly 3-5% a year depending on how it's estimated, but it comes from ten months of data, so I treat it as an observed drift rather than established annual growth. Lagged and smoothed versions of the index added nothing.



Daily mean market_index (orange) against the daily rate level after removing load effects (teal).

5. Model choice

My first model was a single GBM with market_index as a normal feature. It did worse out-of-time than a GBM that ignored the market completely. Between April and July the index is high and so is the season, so the trees learn 'index around 1.3 means mid-year' instead of a smooth price effect. A tree also can't carry a trend past the last date it saw. Adding a date feature made this worse, not better (table below).

So I split the model in two. The GBM handles the load-level structure, which is genuinely non-linear (the distance taper, equipment premiums, regional differences, and quote_signal mattering mostly on short hauls). A linear term on log(rate) takes the daily market index, the daily mean quote_signal and a time trend. The two parts are fitted by backfitting: fit the GBM on the target minus the linear part, refit the linear part on the GBM residuals, and repeat three times. Lanes are described by coordinates instead of city IDs, so the new validation cities borrow from nearby cities instead of falling back to a global average.

The last change I made was adding the daily mean quote_signal to the linear part. Average MAPE went from 2.10% to 2.08%. That is a small gain, about 1% relative, but it improved all five windows, so I kept it. The windows overlap, though, so they aren't five independent confirmations.

Settings: 1,500 trees, learning rate 0.05, 31 leaves, min 20 rows per leaf, early stopping off so runs are reproducible. Things I tried that didn't help or made it worse: L1 loss for the main model, bagging several seeds, a bigger tree size, equipment-specific market slopes, day-of-week terms, and smoothed or lagged market features.

6. Validation results

Average over the five rolling-origin windows (* = final model):

Model	MAPE %	MAE $	RMSE $	Bias %
GBM: load features only	2.62	61.03	96.40	-1.04
GBM: load + market_index as tree feature	3.04	69.76	101.13	-2.29
GBM: load + market + date trend as tree features	3.36	80.58	121.93	-1.39
GBM + linear term, per-load inputs (no same-day averages)	2.09	48.57	77.75	-0.31
GBM + linear market/trend offset *	2.08	47.97	75.66	-0.26

MAPE % by test window:

Test months	GBM + linear term *	GBM + linear, per-load	GBM + market + date	GBM + market	GBM, load only

May-Jun	1.70	1.70	3.25	2.31	2.59
Jun-Jul	1.62	1.63	3.18	2.63	2.30
Jul-Aug	2.89	2.93	4.28	3.19	3.55
Aug-Sep	2.60	2.62	3.75	3.32	3.20
Sep-Oct	1.61	1.59	2.32	3.75	1.47

The final model is best in 4 of 5 windows and has the smallest bias. The one window it doesn't win is Sep-Oct. There the market sat close to its January level, so ignoring it cost nothing. The biggest errors for every model are in Jul-Aug, where there is a level drop around 25 July that the market index doesn't fully explain, and August loads are noisier in general.

Extra checks on the same windows:

Variant (same 5 windows, clean test rows)	**MAPE %**	**MAE $**
Final model	2.08	47.97
Same model, per-load inputs only (no same-day averages)	2.09	48.57
Final model trained with the suspicious rates kept	3.88	86.08

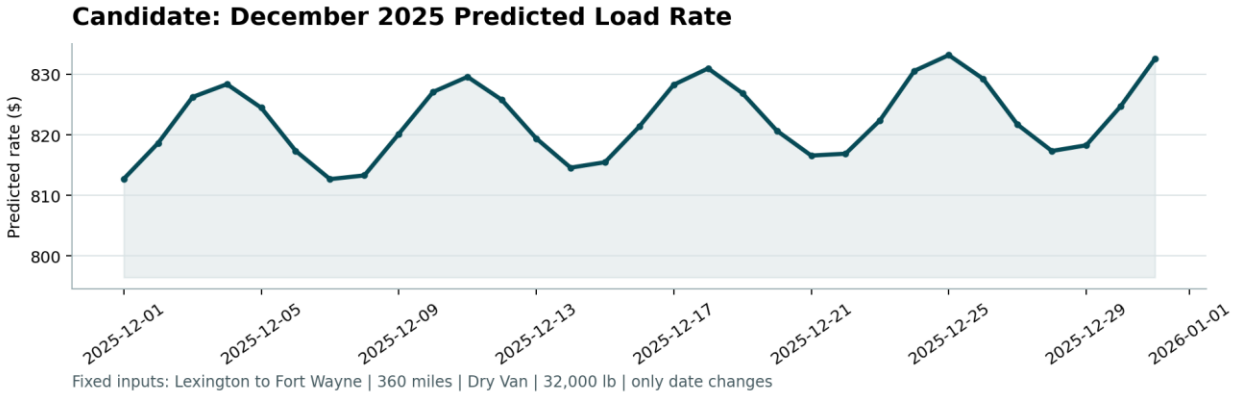
One more diagnostic. To see how much of the remaining error is about getting each day's overall level right, I shifted every test day's predictions by that day's true average error. This uses the test labels, so it can't be done at prediction time and isn't a lower bound on the error. It only brought MAPE to 1.97%, which tells me most of the remaining error is load-to-load variation within a day, not the market level.

Unseen-city check (8 cities removed from training, 3 repeats):

Sep-Oct loads	**MAPE %**	**MAE $**
held-out cities	1.80	41.71
seen cities	1.63	36.65

Loads touching a never-seen city are only slightly worse than the rest, so I'm comfortable predicting the 12% of validation loads that involve one of the new cities.

7. December prediction for the fixed lane (score.py output)



Lexington to Fort Wayne, 360 mi, Dry Van, 32,000 lb, only the date changes.

The predictions run from $813 to $833. For comparison, the dry-van loads on this lane in training were between $758 and $934, with most around $770-$870. The weekly shape (higher mid-week, lower at weekends) comes from the December market index in validation.csv, which follows the same weekly cycle as the rest of the year. The slight rise across the month is the learned trend. Nothing in the training data covers late-December holidays, so the model has no holiday effect, and I'd treat Christmas week as the least reliable part of the chart.

8. What I'd do next

With more time or data I'd look at: a full year of history to learn real seasonality and holidays instead of a linear trend; finding out where the 0.25x / 4x labels come from (if it's a units issue they could be corrected instead of dropped); prediction intervals, since noise varies by month and distance; and retraining every month as new loads come in, because the trend estimate rests on only ten months.